Supporting Procedures in Computer Hardware

Mechanisms in Procedures

- Passing data
  - Procedure arguments
  - Return value
- Passing control
  - To beginning of procedure code
  - Back to return point
- Memory management
  - Allocate during procedure execution
  - Deallocate upon return
- All implemented with machine instructions

```
int poo(...) {
        .
        .
        y = bar(x);
        print(y);
        .
}
```

```
int bar(int i) {
        int t = 3 * I;
        int v[10];
        .
        .
        return v[t];
}
```

Procedure Calling

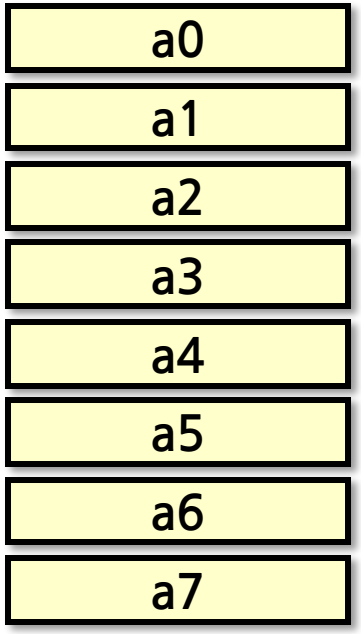
1. Place parameters in registers `x10` to `x17` (or `a0` to `a7`)
2. Transfer control to procedure
3. Acquire storage for procedure
4. Perform procedure's operations
5. Place result in register `x10` (or `a0`) for caller
6. Return to place of call (address in `x1`)

Six Steps in Execution of a Procedure

1. Main routine (caller) places parameters in a place where the procedure (callee) can access them
   - `x10 - x17 (a0 – a7)`: eight argument registers
2. Caller transfers control to the callee
3. Callee acquires the storage resources needed
4. Callee performs the desired task
5. Callee places the result value in a place where the caller can access it
   - `x10 - x11 (a0 – a1)`: two value registers for result values
6. Callee returns control to the caller
   - `x1`: one return address register to return to the point of origin

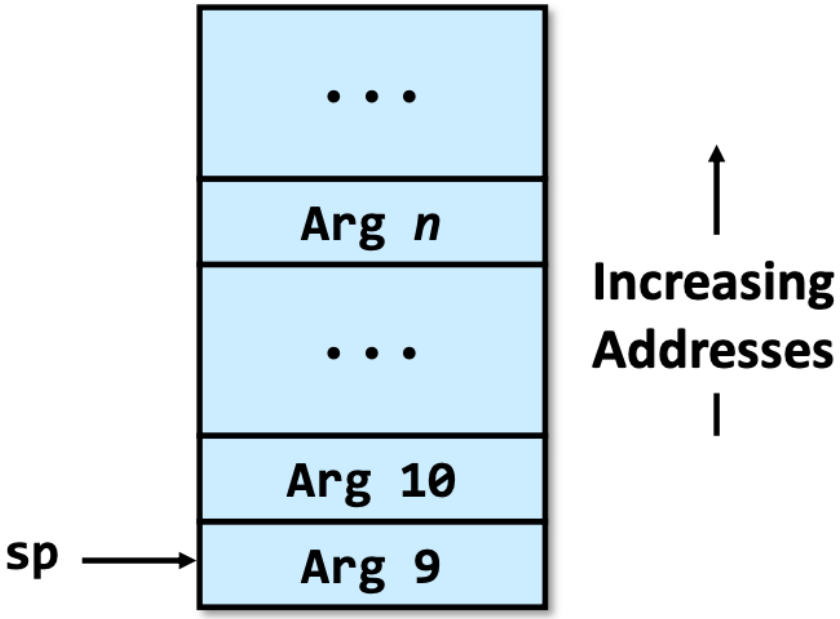
Passing Arguments

- First 8 arguments:



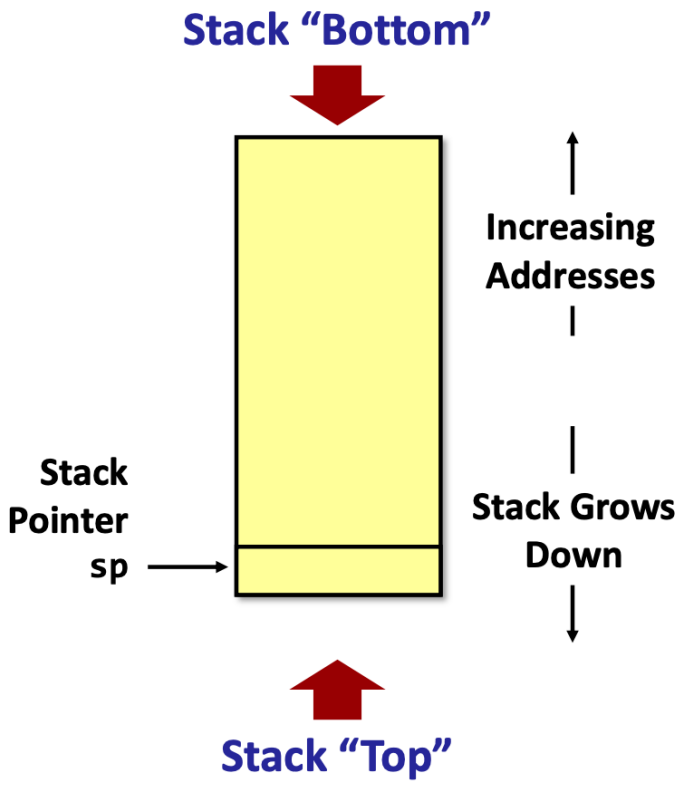
- Return value

- Remaining arguments:
  - Push the rest on the stack in reverse order
  - Only allocate stack space when needed



RISC-V Stack

- Region of memory managed with stack discipline
  - Last-In, First-Out (LIFO)
  - No explicit push/pop operations
  - Load/store instructions used to access stack memory
- Grows toward lower addresses
- Register **`sp`** (**`x2`**) contains lowest stack address
  - Address of "top" element



Procedure Call Instructions

- Procedure call: jump and link

```
jal  x1, ProcedureLabel
```

  - Address of following instruction put in `x1(ra)`
  - Jumps to target address

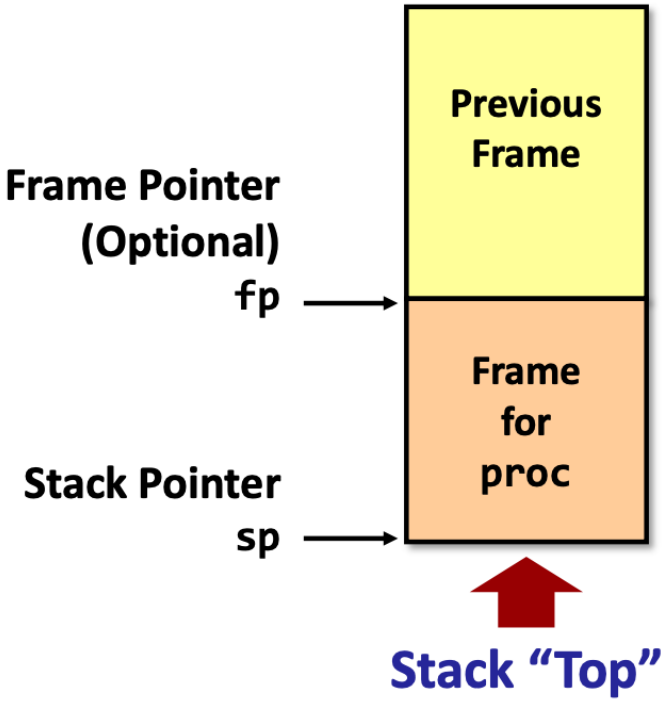
- Procedure return: jump and link register

```
jalr x0, 0(x1)
```

  - Like `jal`, but jumps to `0` + address in `x1(ra)`
  - Use `x0` as `rd` (`x0` cannot be changed)
  - Can also be used for computed jumps
    - e.g., calls via function pointer, virtual method dispatches, etc

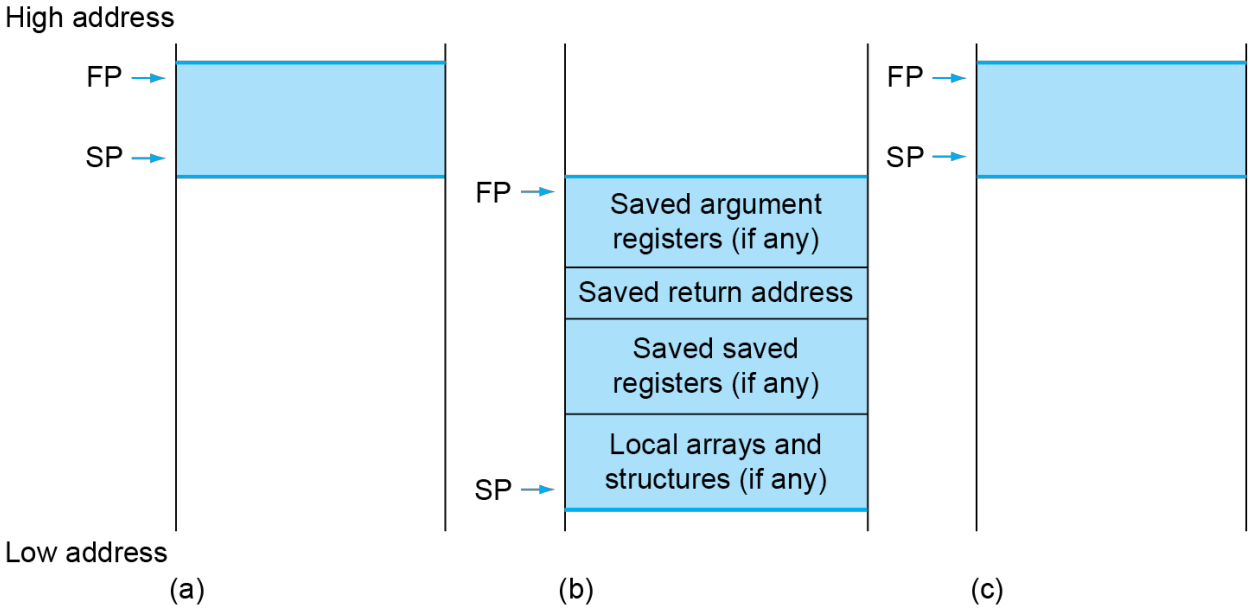
Stack Frame

- Contents
  - Return information
  - Arguments
  - Local variables & temp space
- Management
  - "set-up" code: space allocated when enter procedure
  - "Finish" code: deallocate when return
  - Stack pointer **`sp`** indicates stack top
  - Optional frame pointer **`fp`** indicates start of current frame



Local Data on the Stack

- Local data allocated by callee
  - E.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage



Leaf Procedure Example

- C code:

```
long leaf_example(long g, long h, long i, long j)
{
    long f;
    f = (g + h) - (i + j);
    return f;
}
```

  - Arguments `g, ..., j` in `x10, ..., x13`
  - `f` in `x20`
  - Temporaries `x5`, `x6`
  - Need to save `x5`, `x6`, `x20` on stack

Leaf Procedure Example

- Compiled RISC-V code:

```
leaf_example:
        addi   sp,    sp,    -24
        sd     x5,    16(sp)
        sd     x6,    8(sp)
        sd     x20,   0(sp)
        add    x5,    x10,   x11
        add    x6,    x12,   x13
        sub    x20,   x5,    x6
        addi   x10,   x20,   0
        ld     x20,   0(sp)
        ld     x6,    8(sp)
        ld     x5,    16(sp)
        addi   sp,    sp,    24
        jalr   x0,    0(x1)
```

Save `x5`, `x6`, and `x20` on stack

Procedure body

Result

Restore `x20`, `x6`, and `x5`

Return

Stack “grows” from high address to low address

Register Saving Problem

- When procedure **`yoo()`** calls **`who()`**:
  - **`yoo()`** is the caller, **`who()`** is the callee
- Can register be used for temporary storage?

```
yoo:
    . . .
    addi x5, zero, 419
    jal  ra, who
    add  x5, x5, x6
    . . .
    ret
```

```
who:
    . . .
    add x5, x10, x11
    . . .
    ret
```

  - Contents of register **`x5`** overwritten by **`who()`**

Register Saving Conventions

- "Caller saved" registers
  - Caller saves temporary values in its frame before the call
  - Contents of these registers can be modified as a result of procedure call
  - RISC-V: `x10` - `x17`, `x5` - `x7`, `x28` - `x31`
- "Callee saved" registers
  - Callee saves temporary values in its frame before using
  - Callee restores them before returning to caller
  - The contetens of these registers are preserved acorss a procedure call
  - RISC-V: `x8` - `x9`, `x18` - `x27`

Leaf Procedure Example (Optimized)

- Compiled RISC-V code:

```
leaf_example:
        add   x5,   x10,  x11
        add   x6,   x12,  x13
        sub   x10,  x5,   x6
        jalr  x0,   0(x1)
```

Procedure body

Return

Register	Name	Use	Saver
x5 – x7	t0 – t2	Temporaries	Caller
x10 – x11	a0 – a1	Function arguments/Return values	Caller

Non-leaf Procedures (1)

- Procedures that call other procedures
- For *nested* call, caller needs to save on the stack
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call
- Example C code:

```
long fact (long n)
{
      if (n < 1) return 1;
      else return n * fact(n-1);
}
```

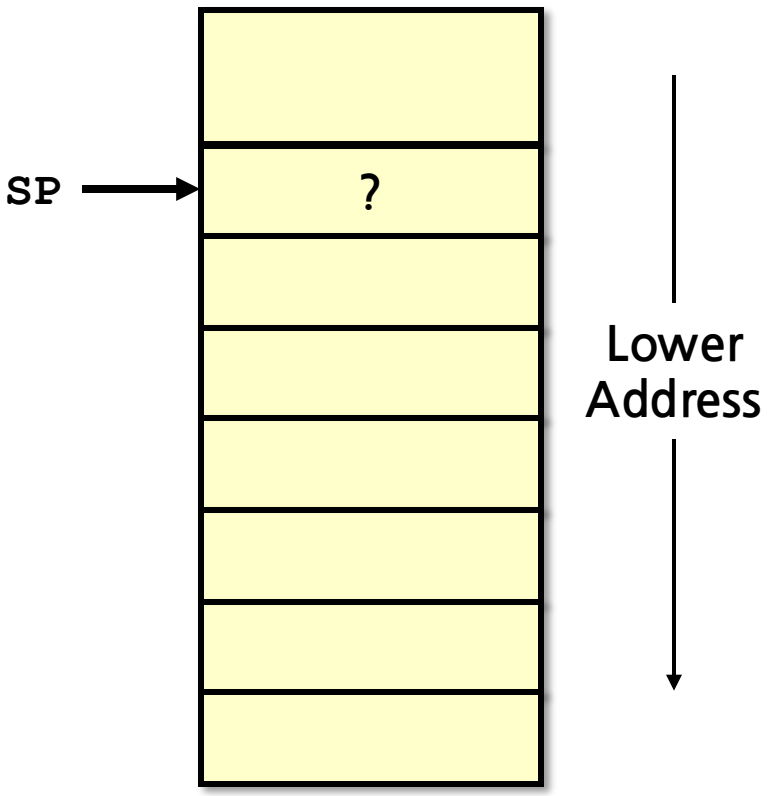
Argument `n` in `a0`
Result in `a10`

Non-leaf Procedures (2)

- RISC-V code:

```
fact:
        addi   sp,   sp,   -16  # adjust stack for 2 items
        sd     ra,   8(sp)      # save return address
        sd     a0,   0(sp)      # save argument (n)
        addi   t0,   a0, -1     # t0 = n - 1
        bge    t0,   zero, L1   # if n >= 1, go to L1
        addi   a0,   zero, 1    # Else, set return value to 1
        addi   sp,   sp, 16     # pop 2 items from stack
        jalr   zero,  0(ra)     # and return
L1:
        addi   a0,   a0, -1     # n = n - 1
        jal    ra,   fact       # recursive call → fact(n-1)
        addi   t1,   a0, 0
        ld     a0,   0(sp)      # restore original n
        ld     ra,   8(sp)      # and return address
        addi   sp,   sp,   16   # pop 2 items from stack
        mul    a0,   a0,   t1   # multiply to get result
        jalr   zero, 0(ra)      # and return
```

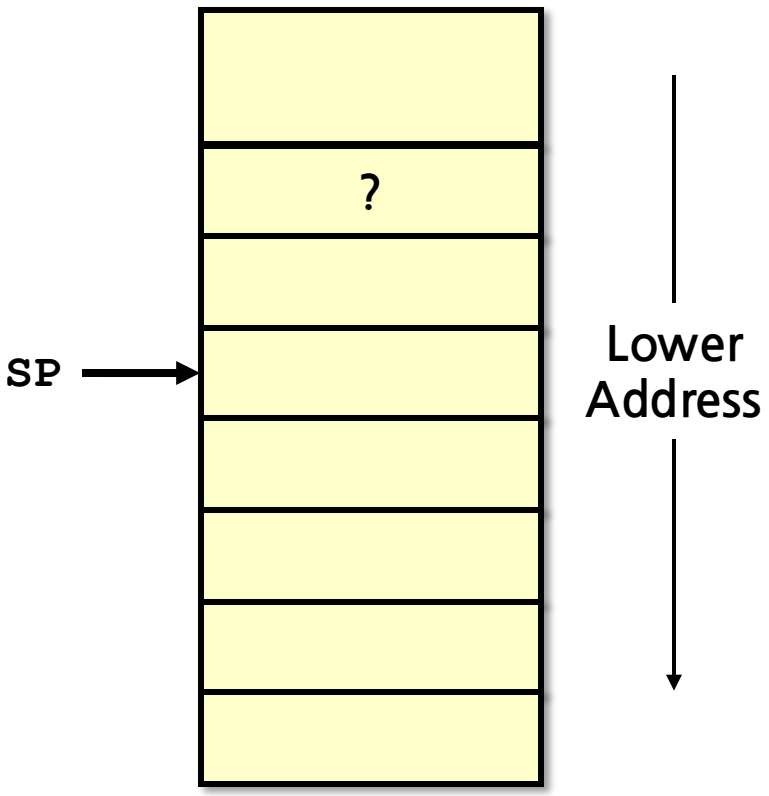
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	?
t1	?

```
fact:
 PC     addi   sp,   sp,   -16
        sd     ra,   8(sp)
        sd     a0,   0(sp)
        addi   t0,   a0, -1
        bge    t0,   zero, L1
        addi   a0,   zero, 1
        addi   sp,   sp, 16
        jalr   zero,  0(ra)
L1:
        addi   a0,   a0, -1
        jal    ra,   fact
        addi   t1,   a0, 0
        ld     a0,   0(sp)
        ld     ra,   8(sp)
        addi   sp,   sp,   16
        mul    a0,   a0,   t1
        jalr   zero, 0(ra)
```

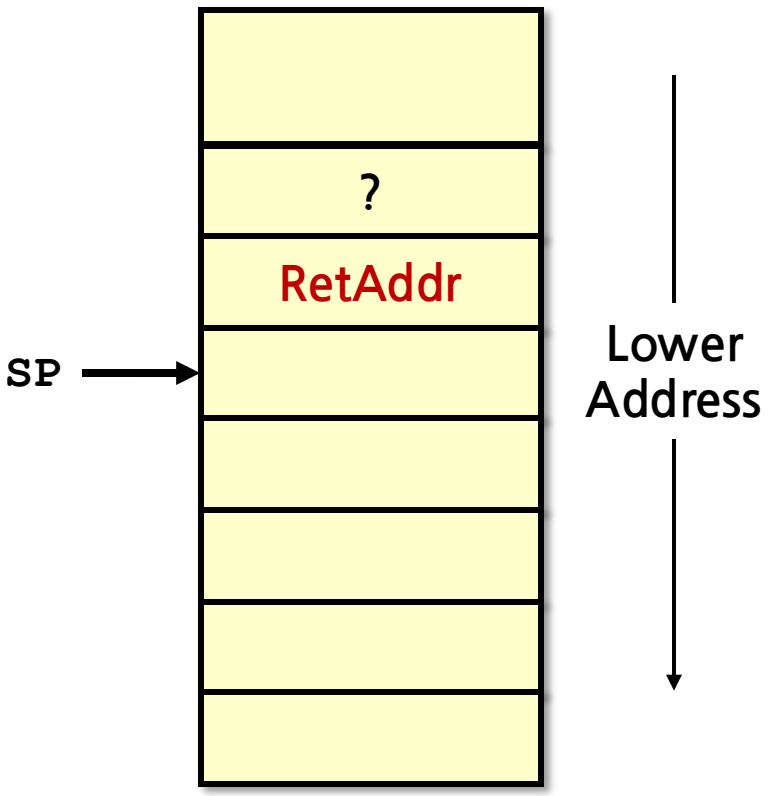
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	?
t1	?

```
fact:
        addi   sp,   sp,   -16
  PC    sd     ra,   8(sp)
        sd     a0,   0(sp)
        addi   t0,   a0, -1
        bge    t0,   zero, L1
        addi   a0,   zero, 1
        addi   sp,   sp, 16
        jalr   zero,   0(ra)
L1:
        addi   a0,   a0, -1
        jal    ra,   fact
        addi   t1,   a0, 0
        ld     a0,   0(sp)
        ld     ra,   8(sp)
        addi   sp,   sp,   16
        mul    a0,   a0,   t1
        jalr   zero, 0(ra)
```

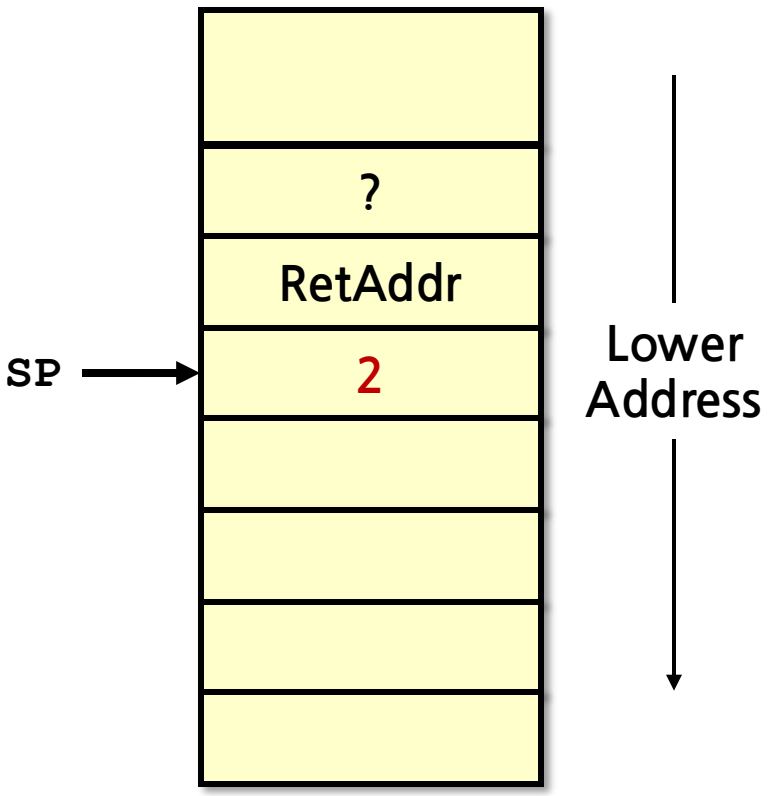
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	?
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
  PC    sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
        addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

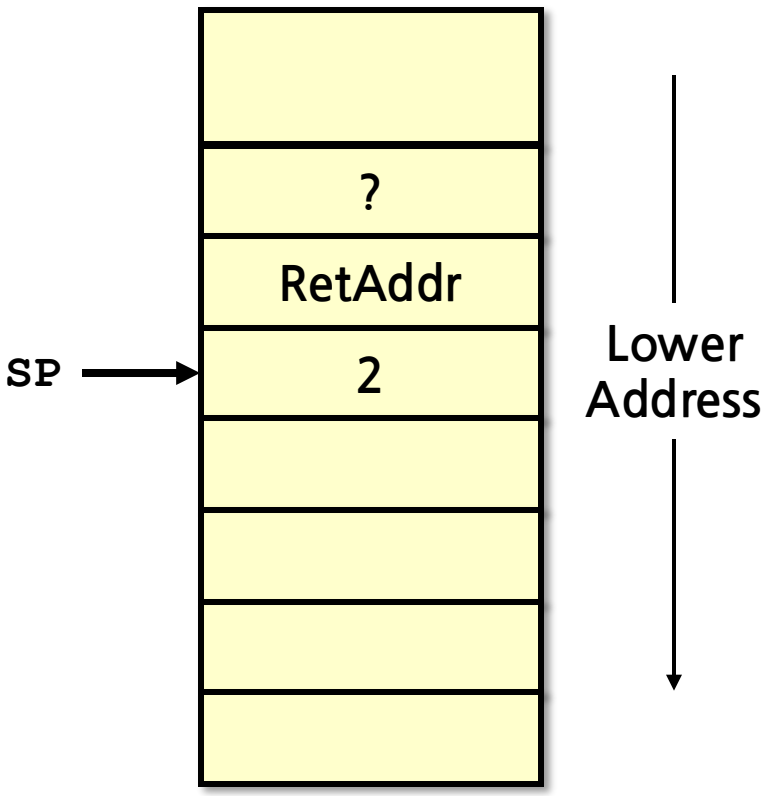
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	?
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
  PC    addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
        addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

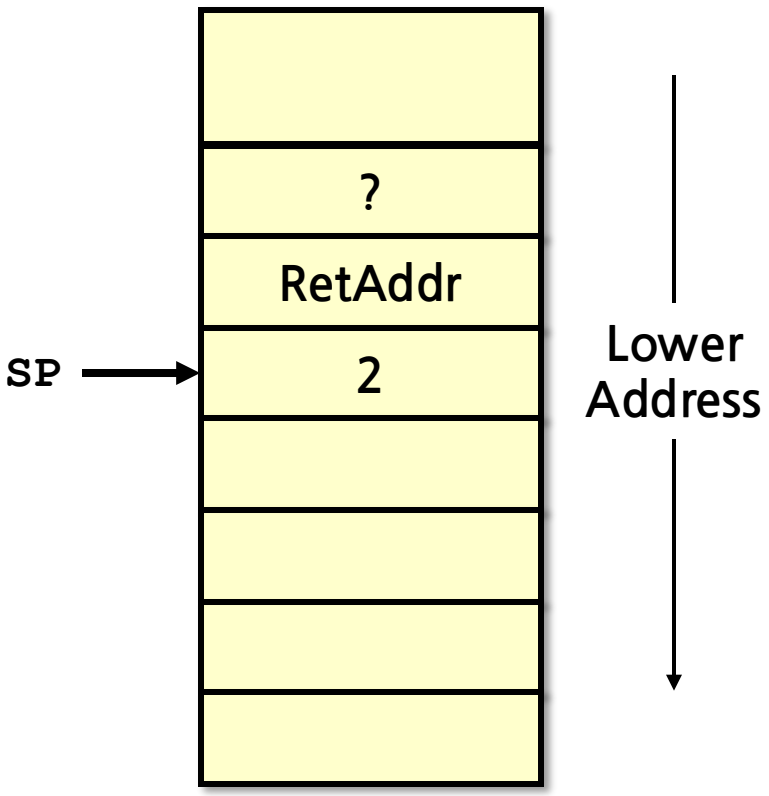
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
  PC    bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
        addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

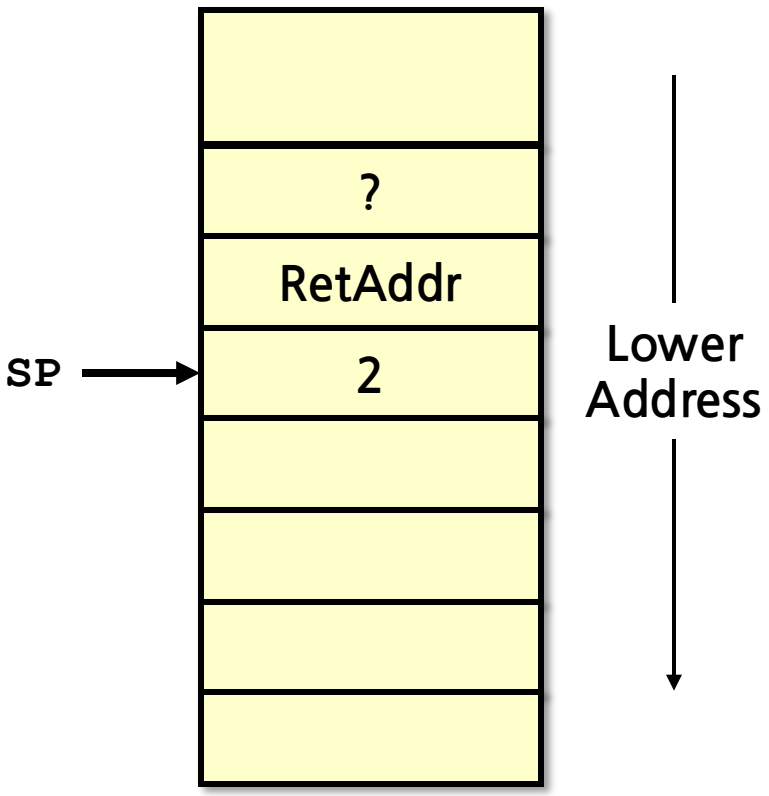
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
  PC    addi    a0,   a0, -1
        jal     ra,   fact
        addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

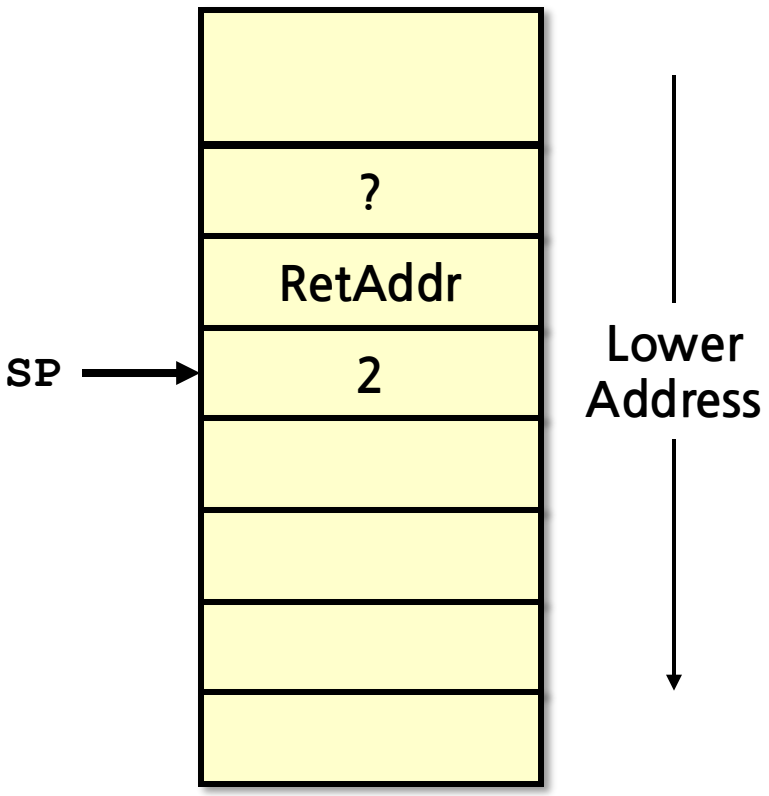
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	1
t0	1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
  PC    jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

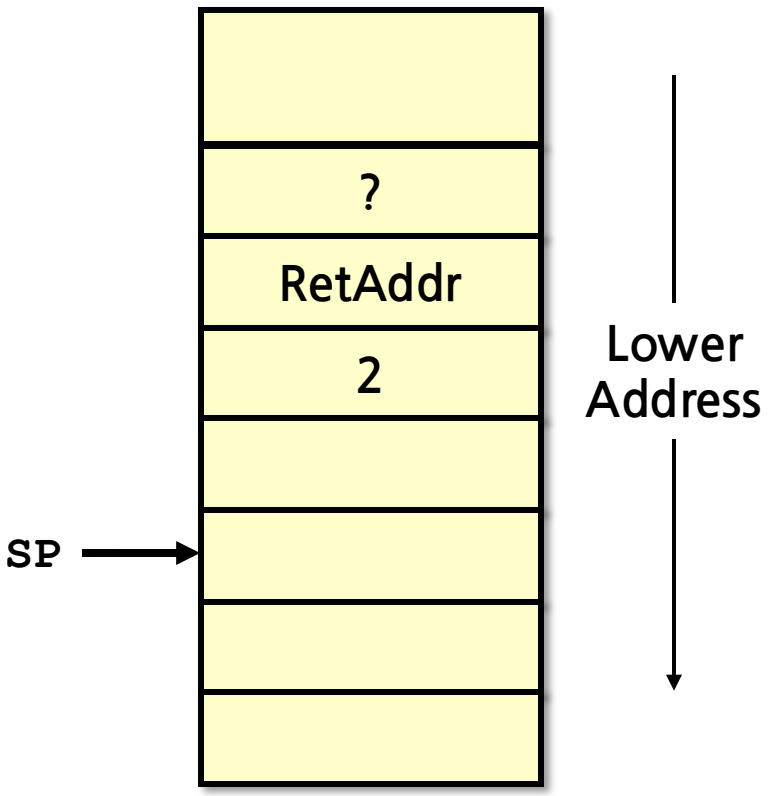
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	1
t1	?

```
fact:
 PC     addi   sp,   sp,   -16
        sd     ra,   8(sp)
        sd     a0,   0(sp)
        addi   t0,   a0, -1
        bge    t0,   zero, L1
        addi   a0,   zero, 1
        addi   sp,   sp, 16
        jalr   zero,  0(ra)
L1:
        addi   a0,   a0, -1
        jal    ra,   fact
A:      addi   t1,   a0, 0
        ld     a0,   0(sp)
        ld     ra,   8(sp)
        addi   sp,   sp,   16
        mul    a0,   a0,   t1
        jalr   zero, 0(ra)
```

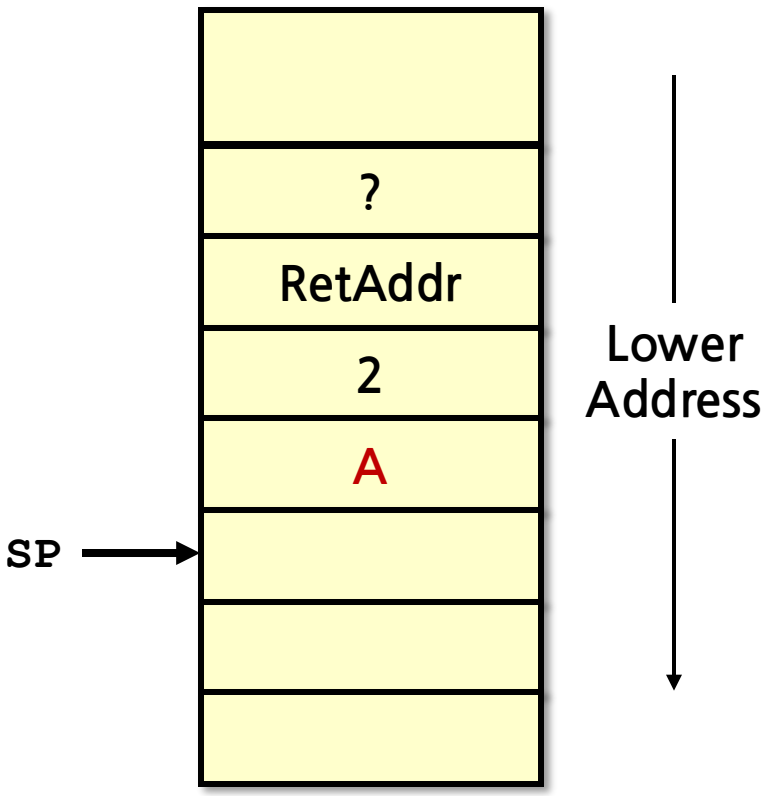
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	1
t1	?

```
fact:
        addi   sp,   sp,   -16
  PC    sd     ra,   8(sp)
        sd     a0,   0(sp)
        addi   t0,   a0, -1
        bge    t0,   zero, L1
        addi   a0,   zero, 1
        addi   sp,   sp, 16
        jalr   zero,  0(ra)
L1:
        addi   a0,   a0, -1
        jal    ra,   fact
A:      addi   t1,   a0, 0
        ld     a0,   0(sp)
        ld     ra,   8(sp)
        addi   sp,   sp,   16
        mul    a0,   a0,   t1
        jalr   zero, 0(ra)
```

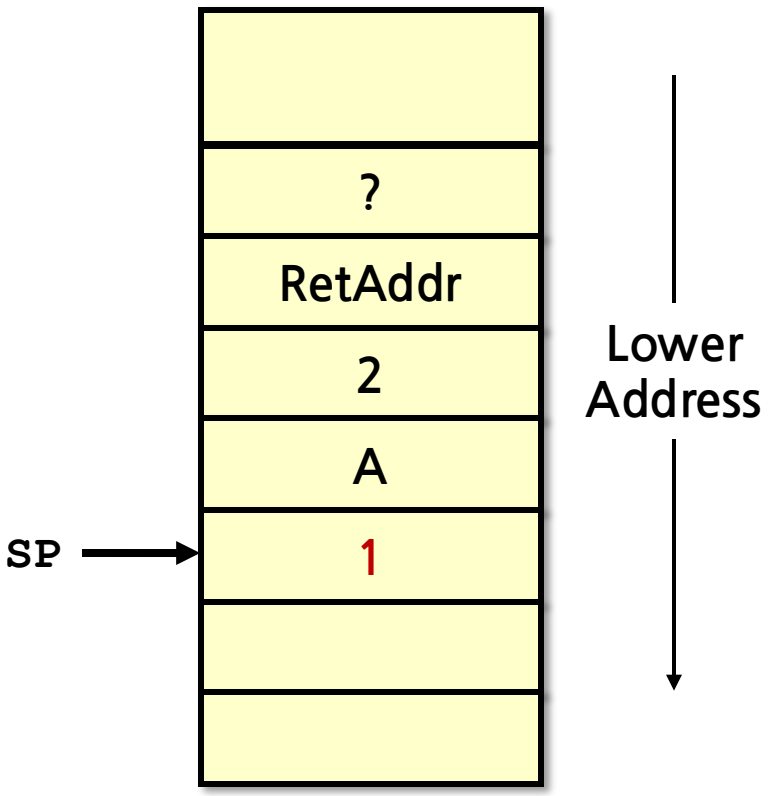
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
  PC    sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

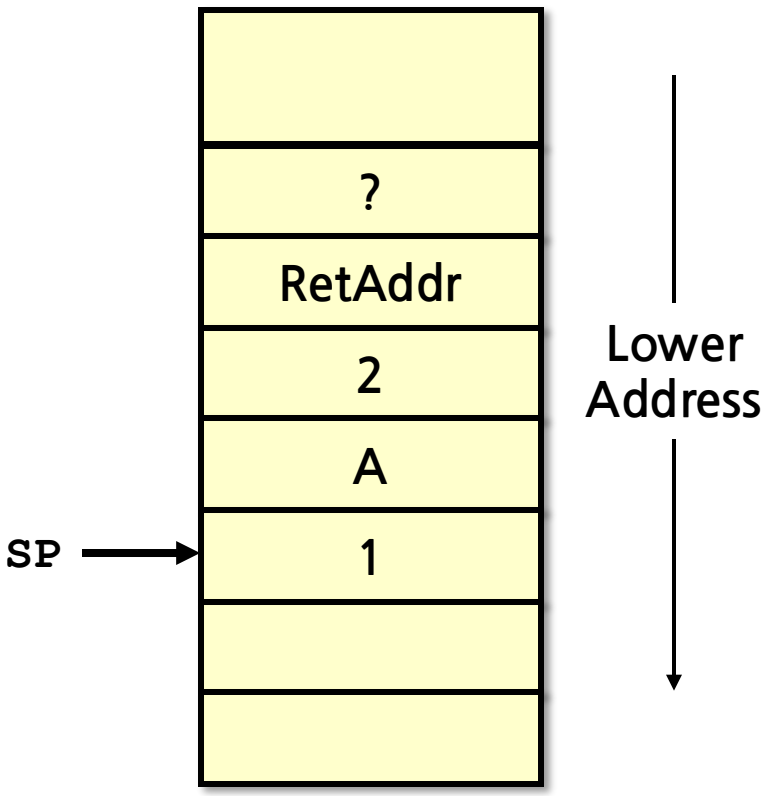
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
  PC    addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

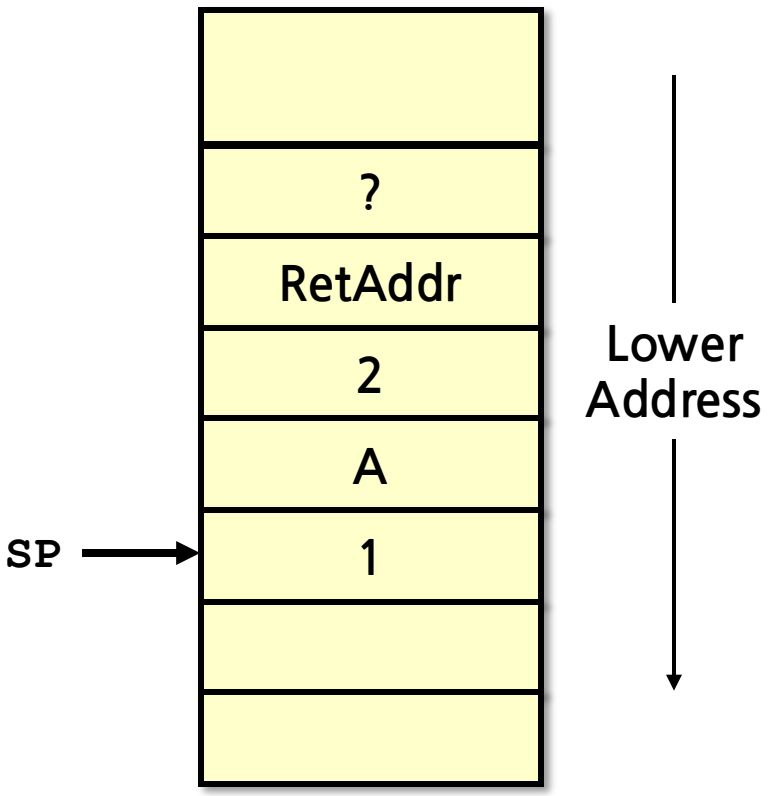
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
  PC    bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

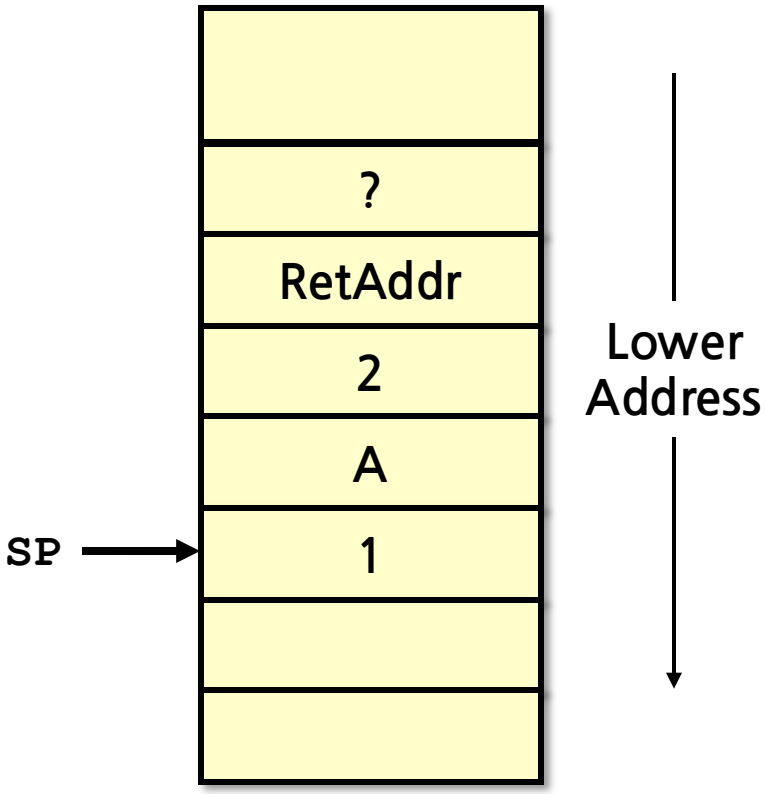
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
  PC    addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

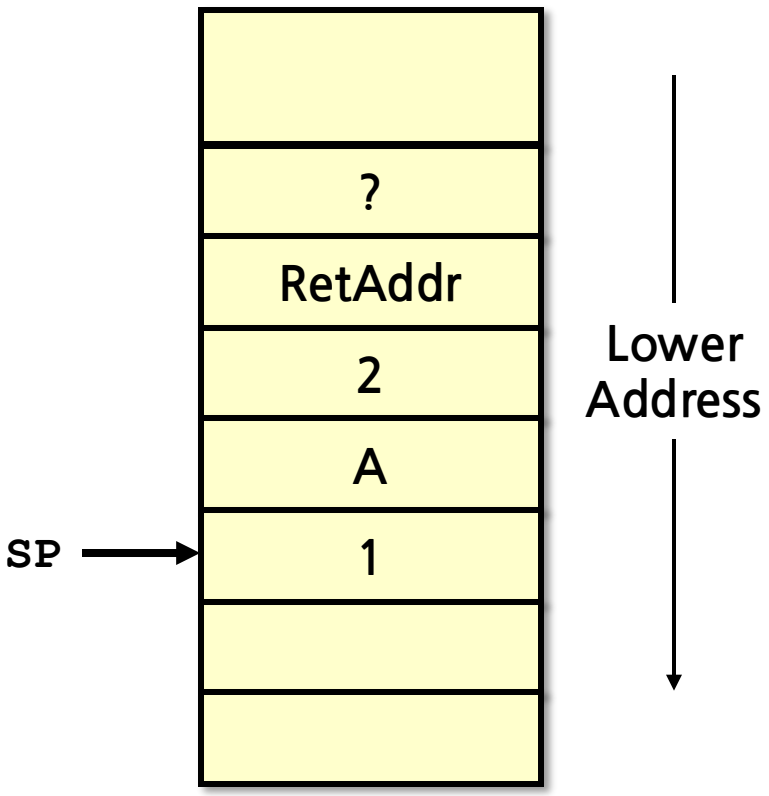
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
  PC    jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

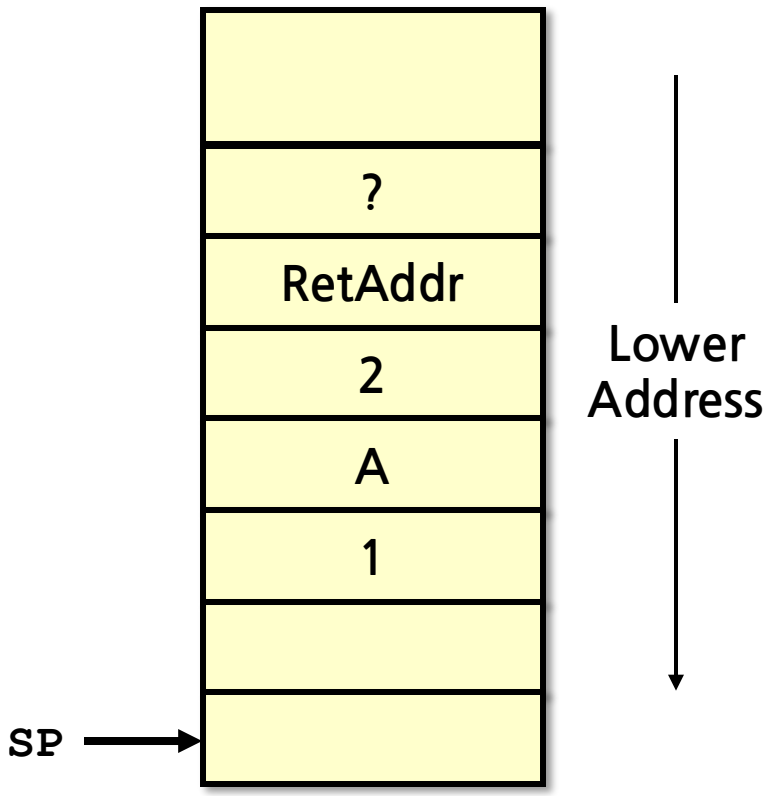
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	0
t1	?

```
fact:
  PC    addi   sp,   sp,   -16
        sd     ra,   8(sp)
        sd     a0,   0(sp)
        addi   t0,   a0, -1
        bge    t0,   zero, L1
        addi   a0,   zero, 1
        addi   sp,   sp, 16
        jalr   zero,  0(ra)
L1:
        addi   a0,   a0, -1
        jal    ra,   fact
A:      addi   t1,   a0, 0
        ld     a0,   0(sp)
        ld     ra,   8(sp)
        addi   sp,   sp,   16
        mul    a0,   a0,   t1
        jalr   zero, 0(ra)
```

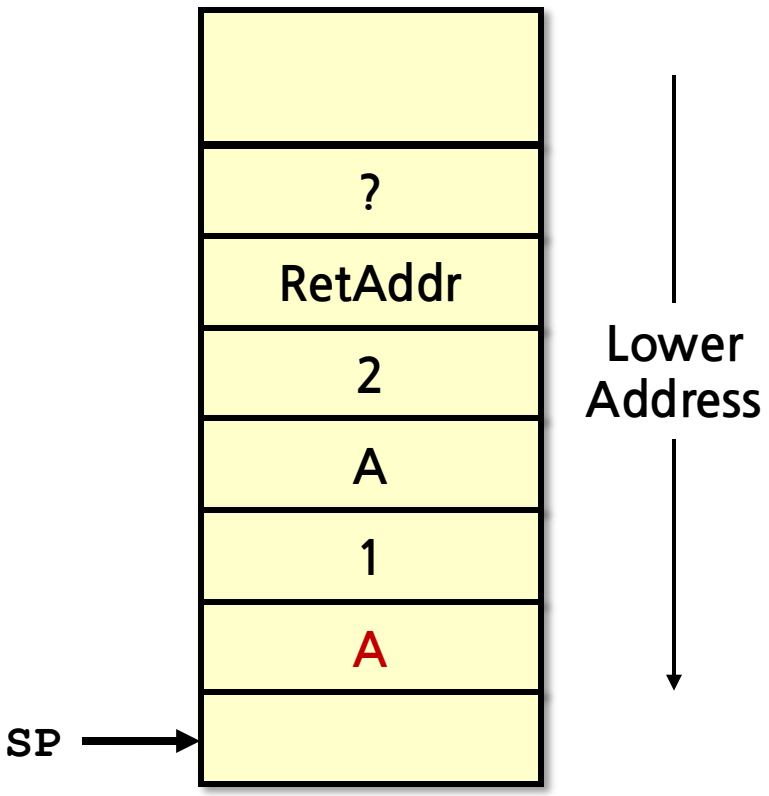
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
  PC    sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

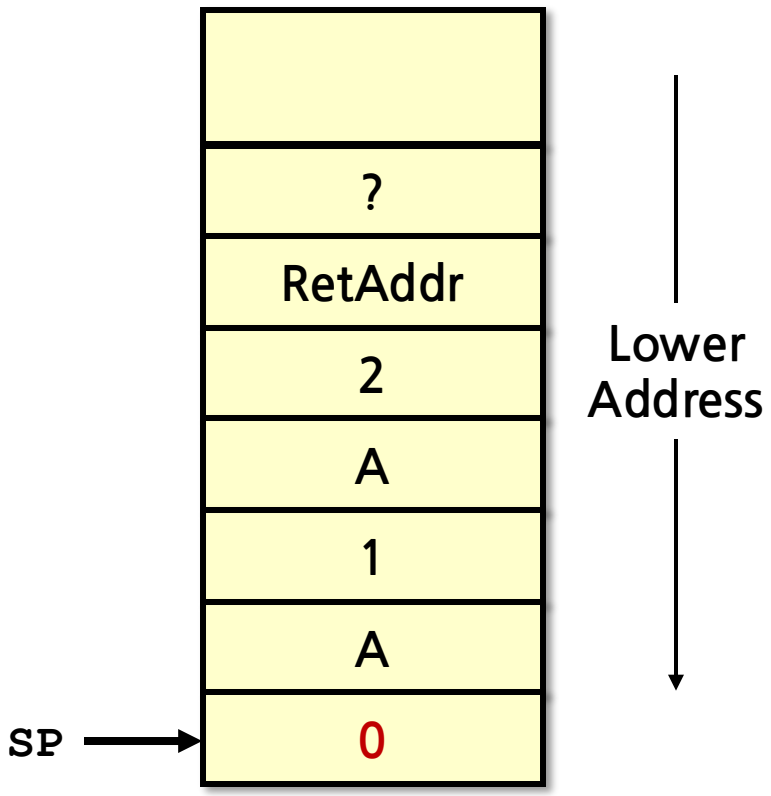
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
  PC    sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

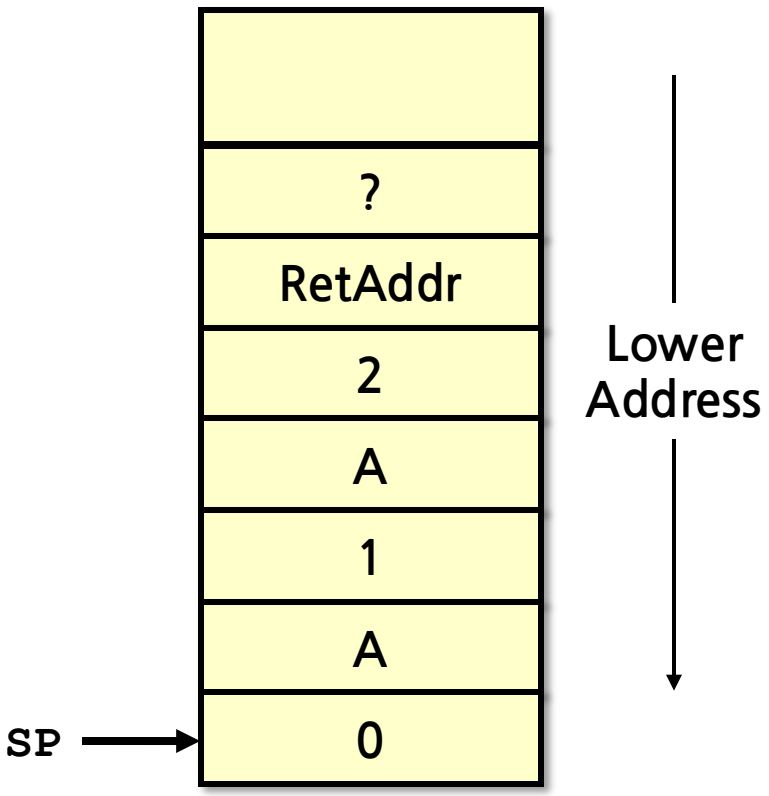
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	0
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
  PC    addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

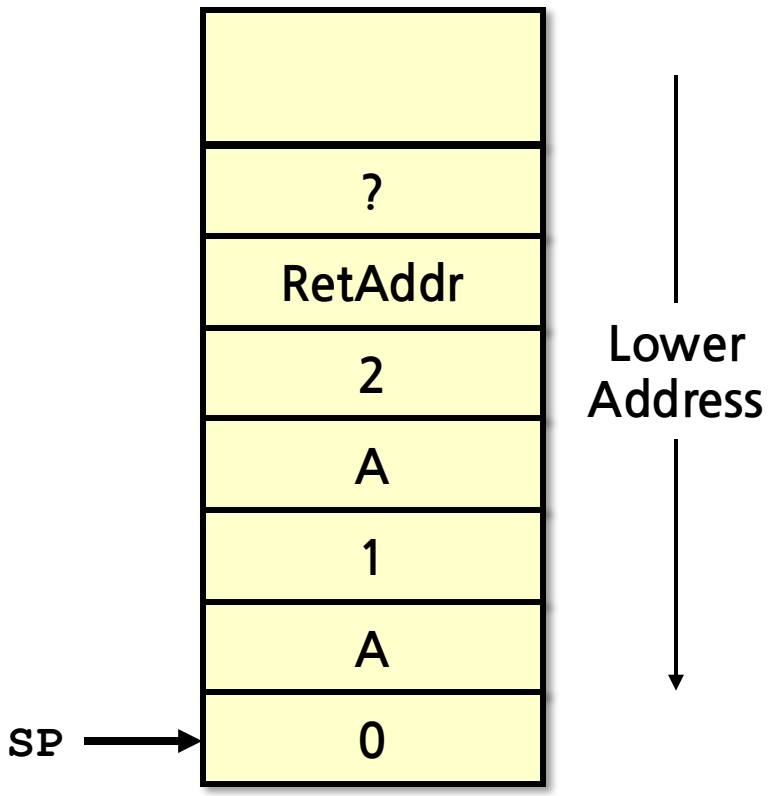
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	-1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
  PC    bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

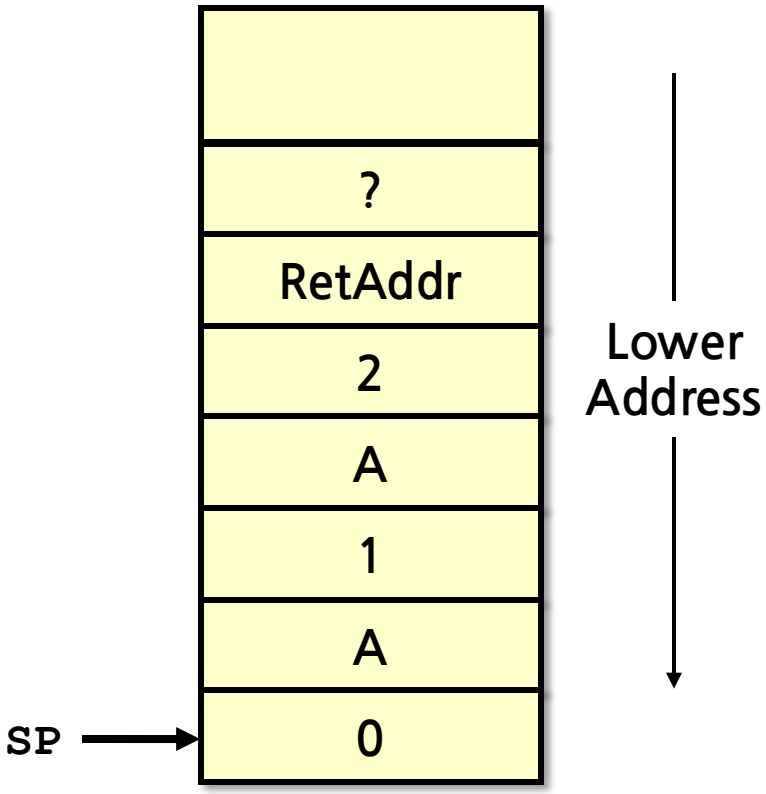
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	0
t0	-1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
PC      addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

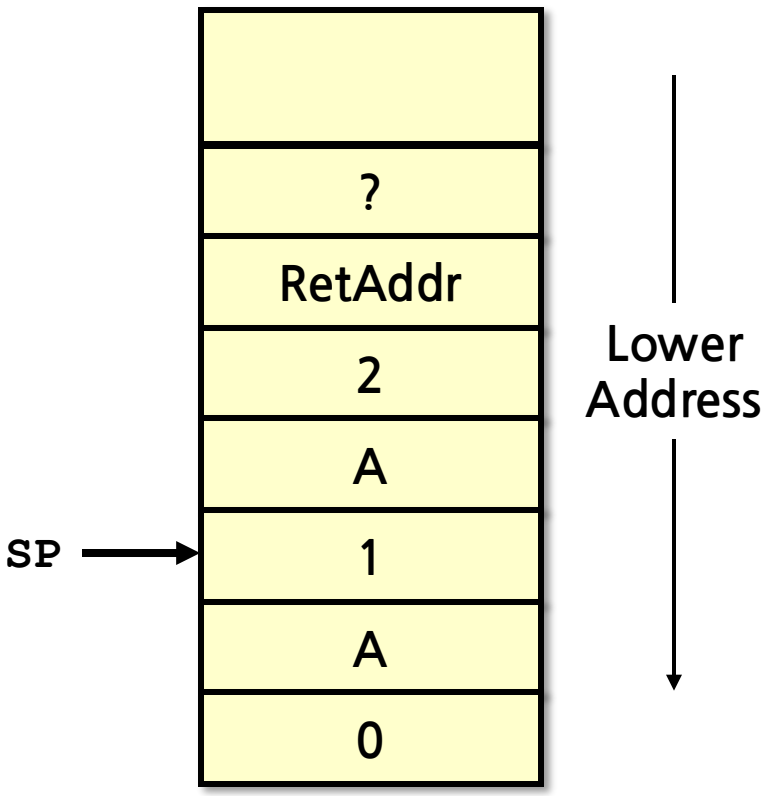
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
  PC    addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

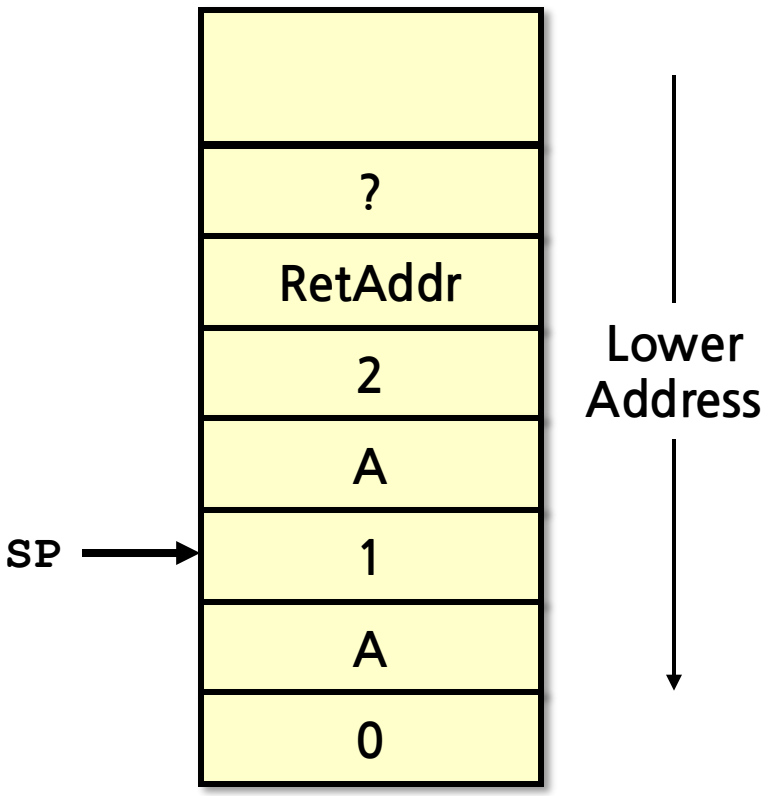
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
  PC    jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

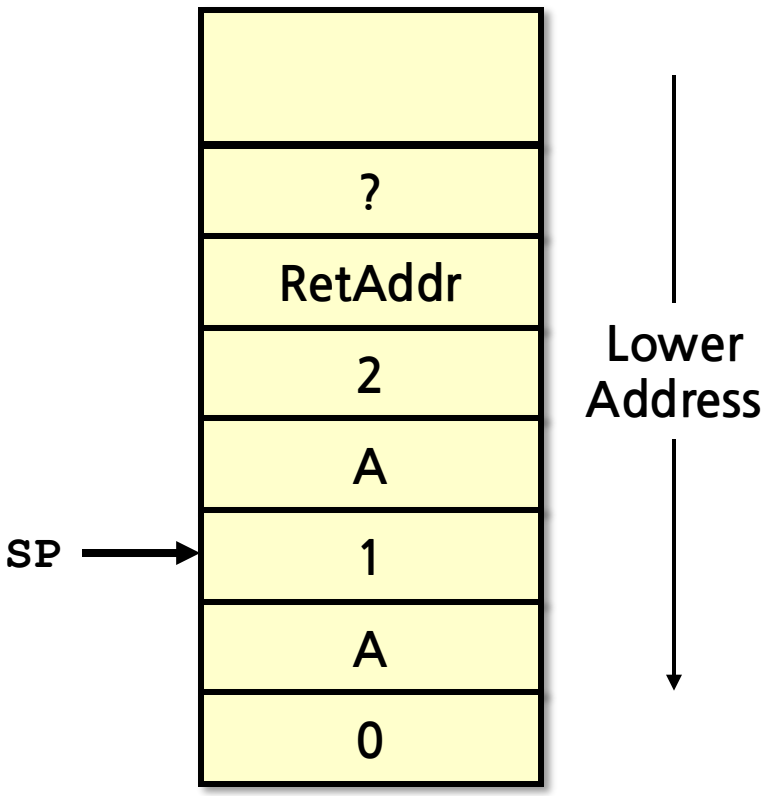
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	?

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
APC     addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

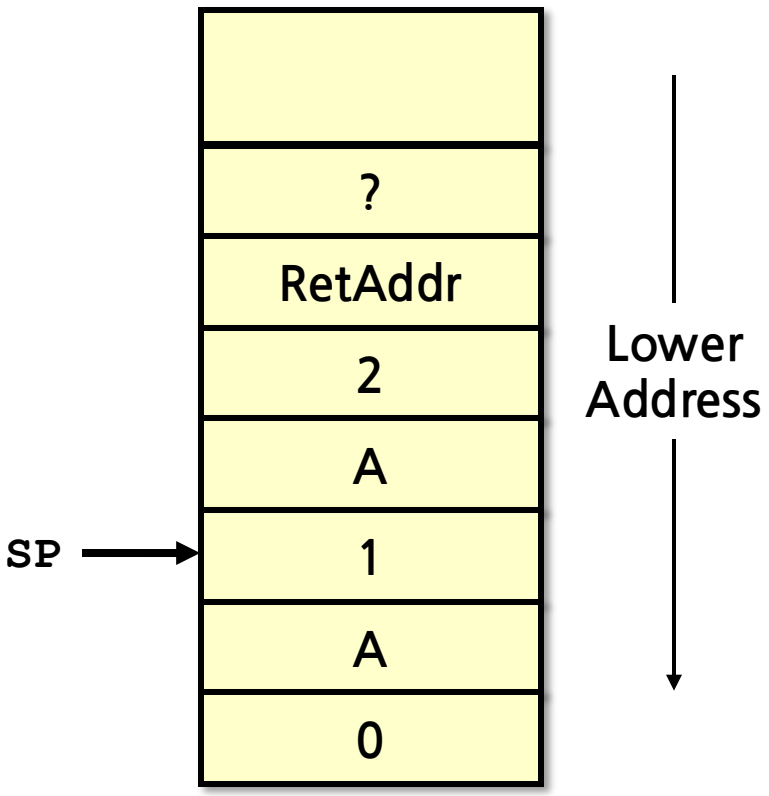
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
  PC    ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

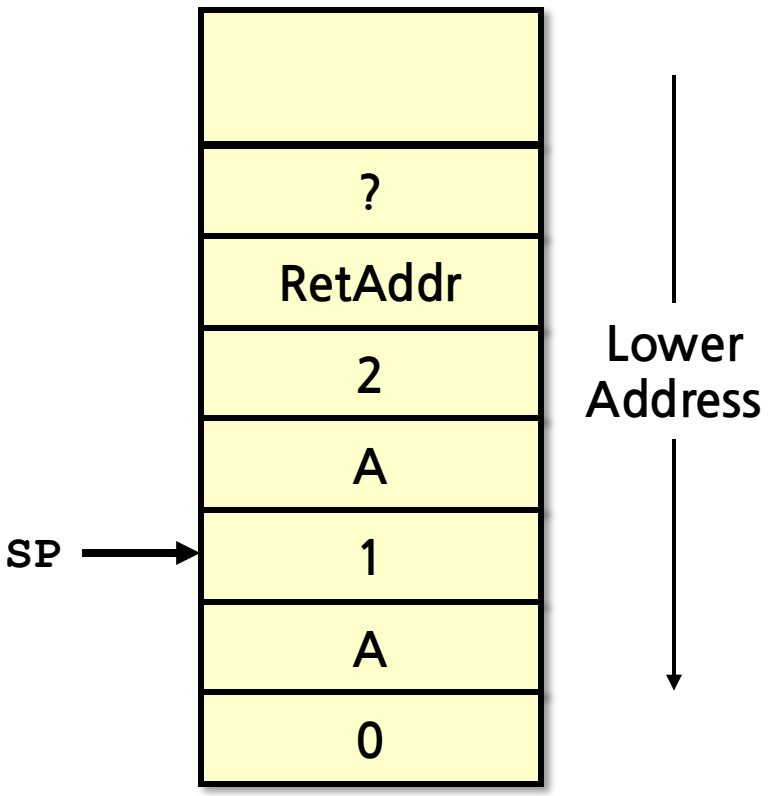
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
  PC    ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

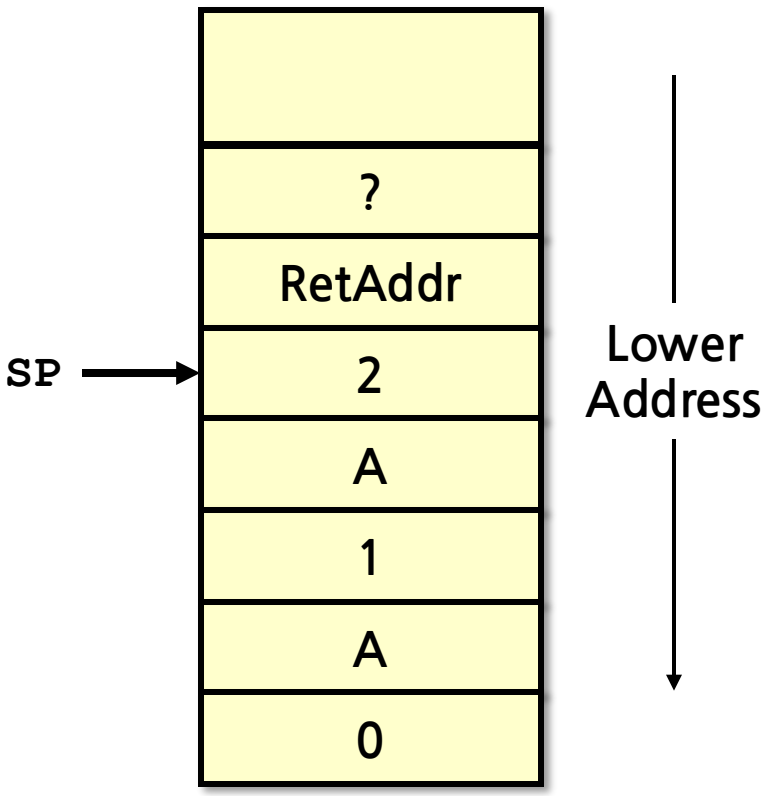
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
  PC    addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

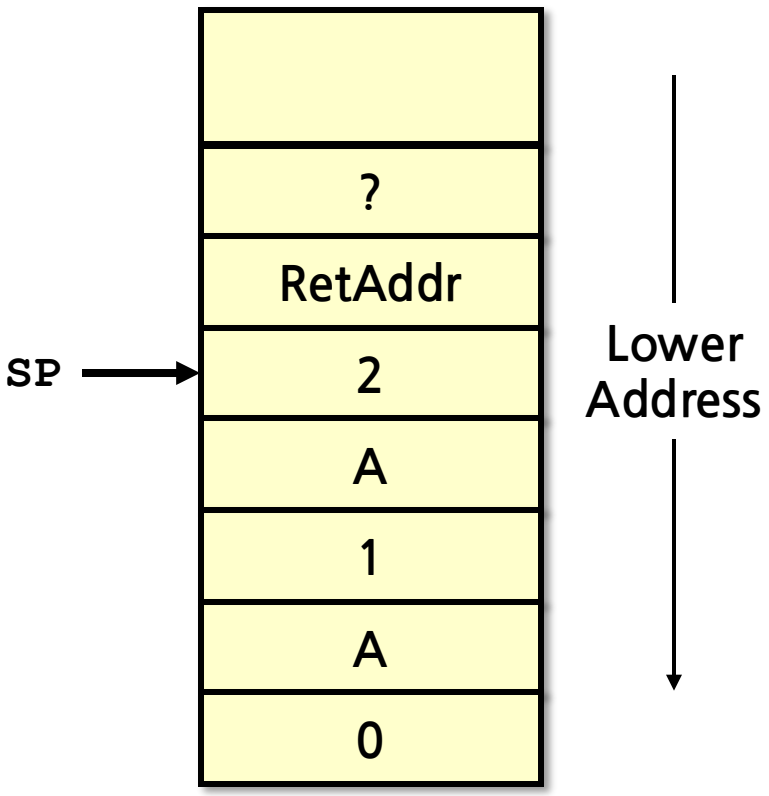
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
PC      mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

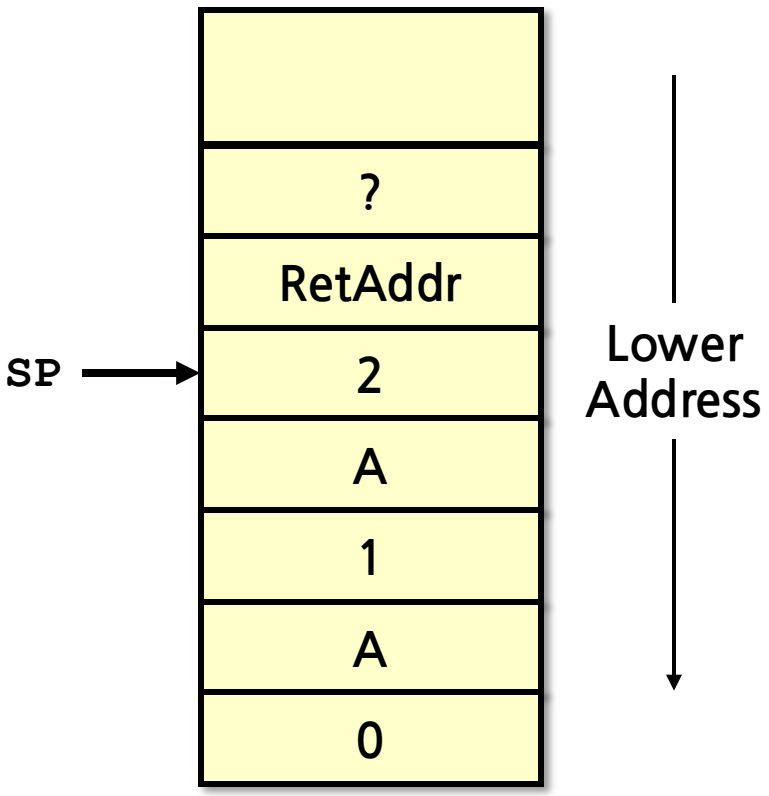
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
PC      jalr    zero, 0(ra)
```

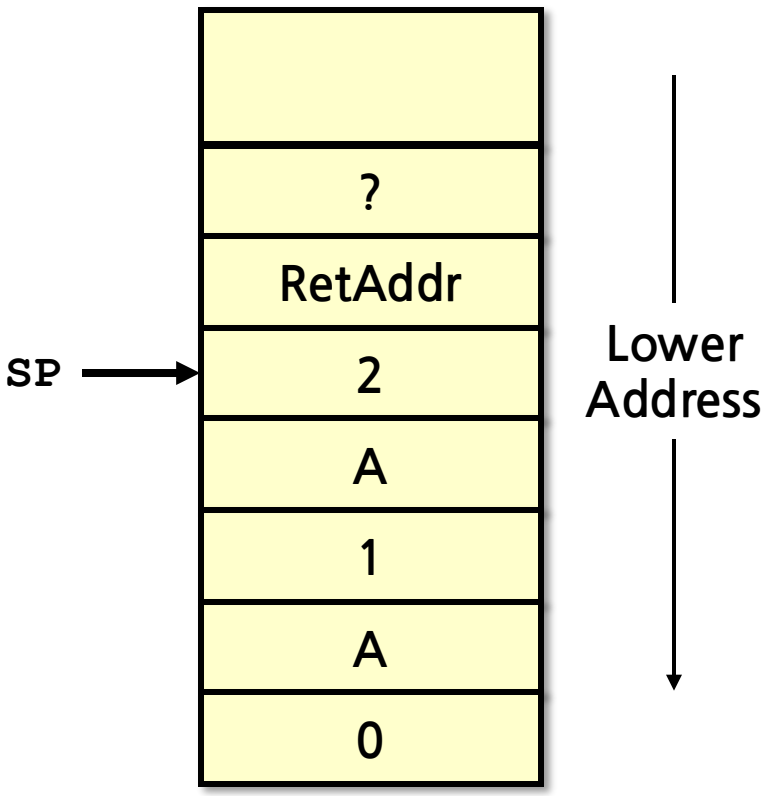
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
APC     addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

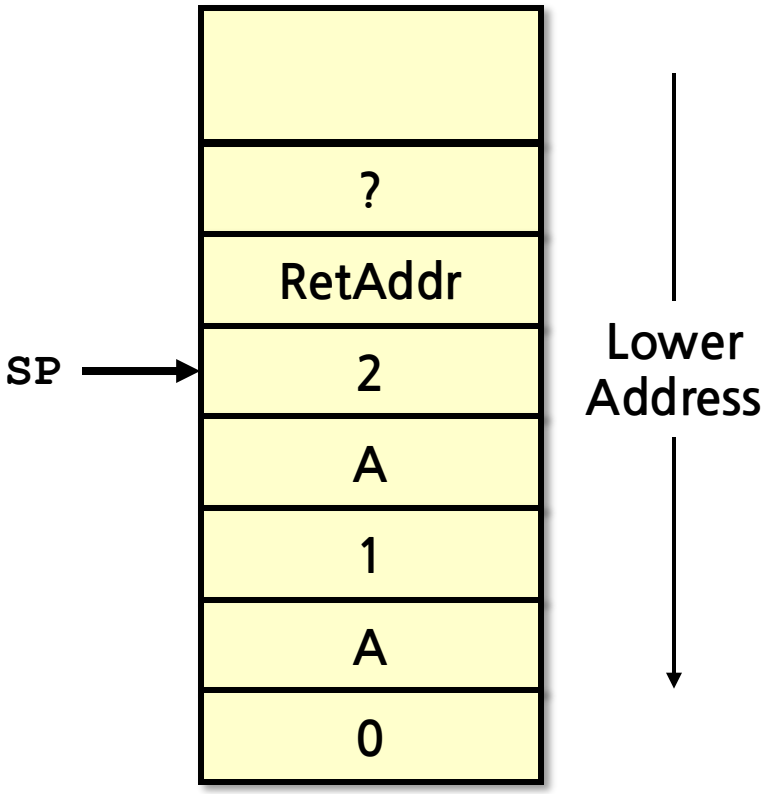
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	1
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
  PC    ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

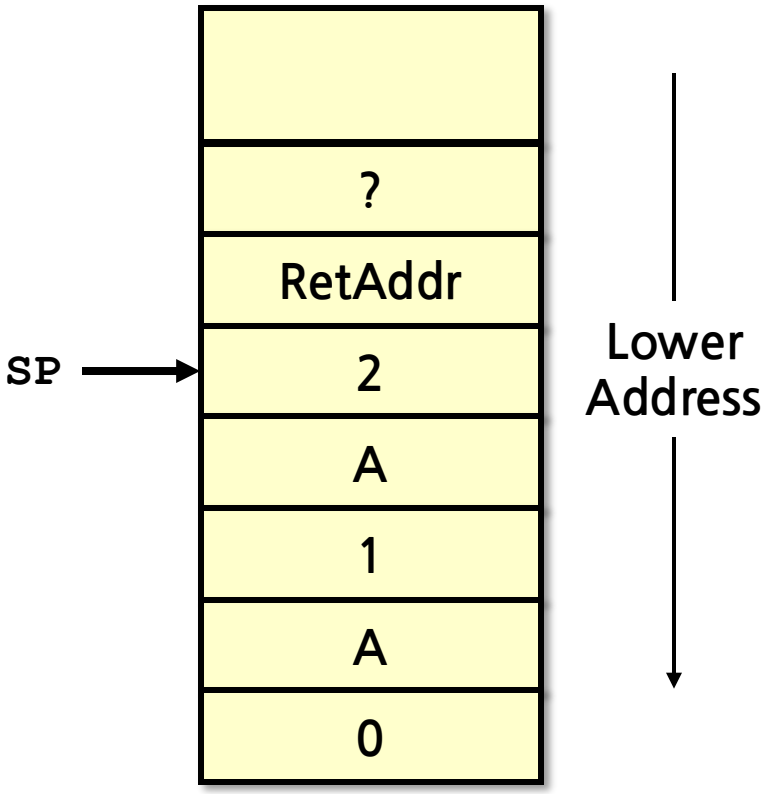
Navigating Stack for `fact(2)`



Register	Value
ra	A
a0	2
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
  PC    ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

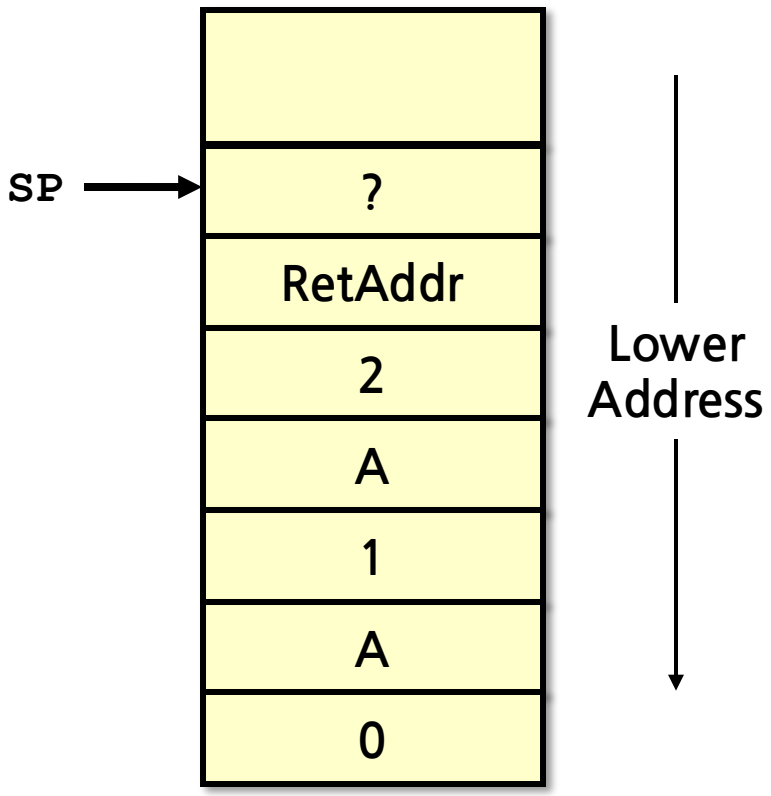
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
  PC    addi    sp,   sp,   16
        mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

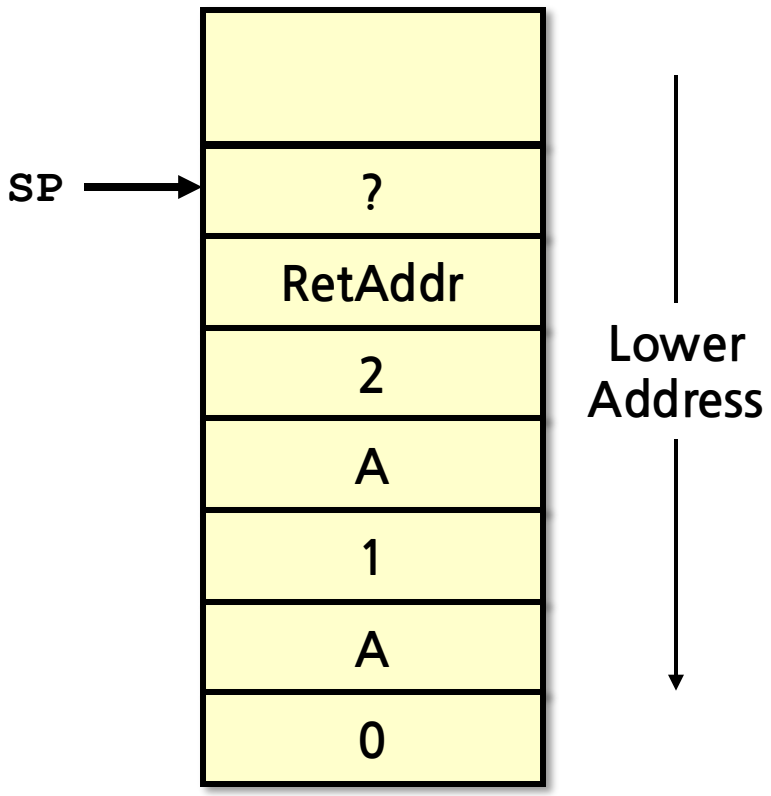
Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
PC      mul     a0,   a0,   t1
        jalr    zero, 0(ra)
```

Navigating Stack for `fact(2)`



Register	Value
ra	RetAddr
a0	2
t0	-1
t1	1

```
fact:
        addi    sp,   sp,   -16
        sd      ra,   8(sp)
        sd      a0,   0(sp)
        addi    t0,   a0, -1
        bge     t0,   zero, L1
        addi    a0,   zero, 1
        addi    sp,   sp, 16
        jalr    zero,   0(ra)
L1:
        addi    a0,   a0, -1
        jal     ra,   fact
A:      addi    t1,   a0, 0
        ld      a0,   0(sp)
        ld      ra,   8(sp)
        addi    sp,   sp,   16
        mul     a0,   a0,   t1
PC      jalr    zero, 0(ra)
```

Recommended Reading

- Chapter 2
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operand of the Computer Hardware
  - 2.5 Representing Instructions in the Computer
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware

Communicating with People (Handling Characters)

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword Operations

- Could use bitwise operations
- MIPS byte/halfword load/store
  - String processing is a common case

```
lb rt, offset(rs)      lh rt, offset(rs)
```

  - Sign extend to 32 bits in rt

```
lbu rt, offset(rs)     lhu rt, offset(rs)
```

  - Zero extend to 32 bits in rt

```
sb rt, offset(rs)      sh rt, offset(rs)
```

  - Store just rightmost byte/halfword

Putting It All Together

Assembler Pseudo Instructions

Pseudo instruction	Base instruction(s)	Meaning
`li rd, imm`	`addi rd, x0, imm`	Load immediate
`la rd, symbol`	`auipc rd, D[31:12]+D[11]` `addi rd, rd, D[11:0]`	Load absolute address where D = symbol - PC
`mv rd, rs`	`addi rd, rs, 0`	Copy register
`not rd, rs`	`xori rd, rs, -1`	One's complement
`neg rd, rs`	`sub rd, x0, rs`	Two's complement
`bgt{u} rs, rt, offset`	`blt{u} rt, rs, offset`	Branch if > (u: unsigned)
`ble{t} rs, rt, offset`	`bge{u} rt, rs, offset`	Branch if ≥ (u: unsigned)
`b{eq|ne}z rs, offset`	`b{eq|ne} rs, x0, offset`	Branch if { = | ≠ }
`b{ge|lt}z rs, offset`	`b{ge|lt} rs, x0, offset`	Branch if { ≥ | < }
`b{le|gt}z rs, offset`	`b{ge|lt} x0, rs, offset`	Branch if { ≤ | > }
`j offset`	`jal x0, offset`	Uncoditional jump
`call offset`	`jal ra, offset`	Call subroutine (near)
`ret`	`jalr x0, 0(ra)`	Return from subroutine
`nop`	`addi x0, x0, 0`	No operation

C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
void swap(long v[], size_t k)
{
  long temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

- `v` in `x10`, `k` in `x11`, `temp` in `x5`

C Sort Example

- Illustrates use of assembly instructions for a C bubble sort function
- Swap procedure (leaf)

```
swap: sll   x6,   x11, 3      # t1 = k * 8
      add   x6,   x10, x6     # t1 = v+(k*4) (address of v[k])
      ld    x5,   0(x6)       # t0 = v[k]
      ld    x7,   8(x6)       # t2 = v[k+1]
      sd    x7,   0(x6)       # v[k] = t2 (v[k+1])
      sd    x5,   8(x6)       # v[k+1] = t0 (temp)
      jalr  zero, 0(x1)       # return to calling routine
```

- `v` in `x10`, `k` in `x11`, `temp` in `x5`

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
          swap(v,j);
        }
    }
}
```

- `v` in `x10`, `n` in `x11`, `i` in `x19`, `j` in `x20`

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
```

```
            li   x19, 0          # i = 0
for1tst     bge  x19, x11, exit1

            << LOOP BODY >>

            addi x19, x19, 1
            j    for1tst
exit1:
```

- `v` in `x10`, `n` in `x11`, `i` in `x19`, `j` in `x20`

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
          swap(v,j);
        }
    }
}
```

- `v` in `x10`, `n` in `x11`, `i` in `x19`, `j` in `x20`

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
```

```
addi  x20, x19, -1
```

- `v` in `x10`,

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
```

```
        addi  x20, x19, -1
for2tst: blt  x20, x0, exit2
```

- v in x10,

The Sort Procedure in C

- Non-leaf (calls swap)

```
void sort (long v[], size_t n)
{
    size_t i, j;
    for (i = 0; i < n; i += 1) {
        for (j = i - 1;
             j >= 0 && v[j] > v[j + 1];
             j -= 1) {
```

```
        addi  x20, x19, -1
for2tst: blt  x20, x0, exit2
        slli  x5, x20, 3
        add   x5, x10, x5
        ld    x6, 0(x5)
        ld    x7, 8(x5)
        ble   x6, x7, exit2
```

- v in x10,

The Procedure Body

```
         mv    x21, x10          # copy parameter x10 into x21
         mv    x22, x11          # copy parameter x11 into x22
         li    x19, 0            # i = 0
for1tst: bge   x19, x11, exit1   # go to exit1 if i >= n
         addi  x20, x19, -1      # j = i - 1
for2tst: blt   x20, x0, exit2    # go to exit if j < 0
         slli  x5, x20, 3        # x5 = j * 8
         add   x5, x10, x5       # x5 = &v[j]
         ld    x6, 0(x5)         # x6 = v[j]
         ld    x7, 8(x5)         # x7 = v[j + 1]
         ble   x6, x7, exit2     # go to exit 2 if (x6 < x7)
         mv    x10, x21          # first swap parameter is v
         mv    x11, x20          # second swap parameter is j
         jal   x1, swap          # call swap
         addi  x20, x20, -1      # j -= 1
         j     for2tst           # jump to test of inner loop
exit2:   addi  x19, x19, 1       # i += 1
         j     for1tst           # jump to test of outer loop
```

Move params

Outer loop

Inner loop

Pass params & call

Inner loop

Outer loop

The Full Procedure

```
sort:    addi sp, sp, -40      # make room on stack for 5 registers
         sd  x1, 32(sp)        # save x1 (return address) on stack
         sw  x22, 24(sp)       # save x22 on stack
         sw  x21, 16(sp)       # save x21 on stack
         sw  x20, 8(sp)        # save x20 on stack
         sw  x19, 0(sp)        # save x19 on stack
         …                     # procedure body
         …
exit1:   ld x19, 0(sp)         # restore x19 from stack
         ld x20, 8(sp)         # restore x20 from stack
         ld x21, 16(sp)        # restore x21 from stack
         ld x22, 24(sp)        # restore x22 from stack
         ld x1, 32(sp)         # restore x1 (return address) from stack
         addi sp, sp, 40       # restore stack pointer
         jalr x0, 0(x1)        # return to calling routine
```

Array vs. Pointers

Arrays vs. Pointers

- Array indexing involves
  - Multiplying index by element size
  - Adding to array base address
- Pointers correspond directly to memory addresses
  - Can avoid indexing complexity

Example: Clearing and Array

```
clear1(long array[], size_t size) {
  size_t i;
  for (i = 0; i < size; i += 1)
    array[i] = 0;
}
```

```
clear2(long *array, size_t size) {
  long *p;
  for (p = &array[0]; p < &array[size];
       p = p + 1)
    *p = 0;
}
```

```
clear1:
       li   t0, 0.         # i = 0
loop1:
       slli t1, t0, 3      # t1 = i * 8
       add  t2, a0, t1     # t2 = &array[i]
       sd   zero, 0(t2)    # array[i] = 0
       addi t0, t0, 1      # i = i + 1
       blt  t0, a1, loop1  # if (i < size)
                           # goto loop1
```

```
clear2:
       mv   t0, a0         # p = & array[0]
       slli t1, a1, 3      # t1 = size * 8
       add  t2, a0, t1     # t2 = &array[size]
loop2:
       sd   zero, 0(t0)    # Memory[p] = 0
       addi t0, t0, 8      # p = p + 8
       bltu t0, t2, loop2  # if (p < &array[size])
                           # goto loop2
```

Comparison of Array vs. Ptr

- Multiply “strength reduced” to shift
- Array version requires shift to be inside loop
  - Part of index calculation for incremented `i`
- Compiler can achieve same effect as manual use of pointers
  - Induction variable elimination
  - Better to make program clearer and safer

Machine-level Programming

- Assembly code is textual form of binary object code
- Low-level representation of program
  - Explict manipulation of registers
  - Simple and explict instructions
  - Minimal concept of data types
  - Many C control constructs must be implemented with multiple instructions